IntroSkills Sample Project

The `IntroSkills` solution contains a sandbox of runnable C# skills for learning about Misty's .NET SDK. This guide walks through the steps required to deploy a C# skill from the `IntroSkills` solution to your Misty II robot. Once you understand the basics, you can explore the sandbox to find examples of how to perform different tasks wih Misty's .NET SDK. Customize these examples by adding your own code, or use them as inspiration to build your own skills for Misty.

You can get the `IntroSkills` solution from the .NET-SDK repository in the Misty Community GitHub organization. Locate the `C#/SampleProjects/IntroSkills/Misty.Skill.IntroSkills.sln` file in your local copy of the repo, and open it in Visual Studio to start exploring the `IntroSkills` sandbox.

The `IntroSkills` solution includes the following projects:

- `SkillLibrary` - A collection of runnable C# skills that you can load into the `IntroSkillsTask` background application and deploy to Misty.
- `IntroSkillsTask` - A Windows IoT background application for deploying C# skills from the `SkillLibrary` project.
- `Simulator` - An example `SimulatedMisty` program that shows how you can use tools from Misty's .NET SDK to simulate events and robot responses, thereby testing your skill code without running it on a Misty II. **Note:** The `Simulator` is currently a work-in-progress, and is not expected to work with all skills. It is included in the `IntroSkills` collection as an exercise for developers interested in experimenting with the mock robot capabilities of the .NET SDK in their own applications.

> **Note:** To install multiple .NET skills on Misty, you must wrap each individual skill in its own uniquely named background application. The `SkillLibrary` project in the `IntroSkills` solution has several `.cs` files, each with code for a different skill. However, each unique background application you deploy can only contain a single .NET skill.
>
> It is not required that you use separate projects for your skill code and your background task wrapper in your own .NET skills. When you use the C# skill template to start writing a new skill, your skill code files and the background application wrapper exist in the same project.

Hello World with .NET

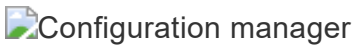
This section provides instructions for deploying the `HelloWorldSkill` from the `IntroSkills` solution. It describes how to configure Visual Studio to deploy .NET skills to Misty, and it demonstrates how to use Misty's .NET SDK to load the code for a skill into the background application file.

Skills you build with Misty's .NET SDK assemble into IoT background applications that run alongside Misty's software on Windows IoT Core. When Misty initiates a .NET skill, her system launches the corresponding background application, which instantiates the interface used to invoke commands and access robot data in your skill code. You can learn how to set up a background application from scratch in the Microsoft developer documentation. For this guide, however, we can use the `IntroSkillsTask` project included in the sample solution.

1. Configuring the Solution

Follow the steps below to configure Visual Studio to deploy .NET skills from the `IntroSkills` project to Misty.

1. Open the `IntroSkills` solution in Visual Studio. This solution includes three projects: **`IntroSkillsTask`**, **`SkillLibrary`**, and **`Simulator`**. This guide is only concerned with the `IntroSkillsTask` and `SkillLibrary` projects.
2. The `IntroSkillsTask` project contains the code for the background application that runs your .NET skill. Set this project as the startup project by right-clicking the project's name in your **Solution Explorer** and selecting **Set As Startup Project**.
3. Use the drop-down menus at the top of Visual Studio to set your solution configuration to **Debug** and the solution platform to **ARM**.
4. Next, follow these steps to configure Visual Studio to deploy the application to Misty:
    1. Right-click the `IntroSkillsTask` project in the **Solution Explorer** and select **Properties**.
    2. Select **Debug** from the left navigation menu.
    3. From the **Target device** menu, select **Remote Machine**.
    4. In the **Remote machine** field, enter Misty's IP address. **Important!** If you have not manually applied Misty's 2020.01.07 Android patch, then you must use the IP address for Misty's USB-to-Ethernet adapter in order to attach a remote debugger. If you have applied the patch, you can attach a debugger using Misty's Wi-Fi IP address (supplied in Misty Connect).
    5. From the **Authentication mode** menu, select **Universal (Unencrypted Protocol)**. Save your changes. The configuration properties should look similar to the image below, with your own robot's address in the Remote machine field:

        
        Configuration manager

        .

2. Installing Misty's .NET SDK Libraries

Misty's .NET SDK includes three libraries for developing .NET skills. They are:

- MistyRobotics.Common
- MistyRobotics.SDK
- MistyRobotics.Tools (optional)

You must install these libraries using Visual Studio before you can build and deploy .NET skills from the `IntroSkills` project. Follow these steps to do so:

1. Right-click the `IntroSkillsTask` project in the **Solution Explorer** and select **Manage NuGet Packages**.
2. Select **nuget.org** as the package source. Make sure the **Include prerelease** box is checked.
3. Search for **misty robotics**. Install the **MistyRobotics.Common** and **MistyRobotics.SDK** libraries.

3. Setting Up the Background Application

With the SDK libraries installed, we're ready to update the code files and deploy the `HelloWorld` skill.

Start by opening the `StartupTask.cs` file from the `IntroSkillsTask` project. This file is the background application wrapper for our .NET skill. Notice that it uses the following resources:

```
using SkillLibrary; // Enables app to use skill code
using MistyRobotics.Common.Types; // Enables app to use robot data types
using MistyRobotics.SDK.Messengers; // Enables connection to robot
```

```
using Windows.ApplicationModel.Background; // Enables app to schedule
background tasks
```

C#

Next, lets walk through the different components of the `StartupTask.cs` file. The `IntroSkillsTask` namespace becomes the name of the background application deployed to Misty. You can change this name to anything you like. This name is **not** required to match the name of your skill, and it does **not** appear in the Skill Runner interface.

When Misty starts up the `IntroSkillsTask` background application, the `Run()` method is called. In the `Run()` method, we use `RobotMessenger.LoadAndPrepareSkill()` to register events and instantiate the robot interface, and we pass in an instance of the background task and the skill we want to run. Additionally, we can specify the log level and set a logging preface, which makes it easier to identify the messages this skill writes to Misty's log files.

```
namespace IntroSkillsTask
{
    /// <summary>
    /// Called upon deploy and when the robot attempts to connect to
    /// the skill
    /// </summary>
public sealed class StartupTask : IBackgroundTask
{
    public void Run(IBackgroundTaskInstance taskInstance)
    {
        // Call LoadAndPrepareSkill to register robot events and
        // add an instance of the robot interface to the skill
        RobotMessenger.LoadAndPrepareSkill
        (
            // Background task instance passed in for task
            // management, managed by system
            taskInstance,

            // Creates a new instance of the skill
            new MostlyHarmlessSkill(),

            // Sets skill log level
            SkillLogLevel.Verbose,

            // Overwrites default logging preface to help label
            // runs of this skill in log files
            "Skill Test Run #1 => "
        );
    }
}
```

C#

By default, the `IntroSkillsTask` application is configured to run the `MostlyHarmlessSkill` from the `SkillLibrary` project. To load a different skill into this application, pass the name for a different skill file from the `SkillLibrary` project into the `RobotMessenger.LoadAndPrepareSkill()` method.

Replace the `new MostlyHarmlessSkill()` argument with `new HelloWorldSkill()`:

```
// Creates a new instance of the skill
new HelloWorldSkill(),
```

C#

4. Deploying the HelloWorldSkill

With the above steps completed, we're ready to deploy the skill to Misty. Follow these steps:

1. Click **Run/Deploy** in Visual Studio, and wait for the skill to deploy. Deployment can take a few minutes. Check the Visual Studio console to know when deployment is complete.

2. After deployment, you may need to reload Misty's skills before you can start the .NET skill from the Skill Runner web page. Reloading is also required anytime you make changes to a .NET skill and re-deploy it to Misty. To reload skills, send a POST request to the API Endpoint for the `ReloadSkills` operation in Misty's REST API: `POST <robot-ip>/api/skills/reload`. (The .NET-SDK repository includes a simple HTML page (`ReloadSkills.html`) that you can load in your web browser to help with this call.)
3. Open up the Skill Runner web page in your browser and connect to your robot. Your .NET skill should appear in the Manage Section of the page. Click **Start** to start running the skill.

> **Note:** If you want to see and run multiple skills at the same time, you must wrap each skill in a different background task. The `IntroSkills` example solution is currently structured such that you can only load one skill into the background application at a time. To install all skills from the `SkillLibrary` on your robot, you may choose to create a new project or solution with a uniquely named background task for each skill in the library.

Now that you understand how to deploy a skill from `IntroSkills` sandbox, you can follow these same steps to run any skills from the `SkillLibrary` project on your robot.

SkillLibrary

This section describes the function of each skill in the `SkillLibrary` sample project. You can load any of these skills into the `IntroSkillsTask` background application to deploy them to Misty, or you can copy the `SkillTemplate` file to build your own skill from scratch.

SkillTemplate

A simple template for developing .NET skills in C#. This template implements the `IMistySkill` interface so that you can invoke commands and use data from Misty. It provides an interface for declaring meta information about the skill, and it instantiates methods you can populate with your code to determine how Misty behaves when the skill starts, pauses, and cancels.

MostlyHarmlessSkill

On start, this skill loops to randomly change Misty's chest LED until the skill is cancelled or times out.

MostlyHarmlessTooSkill

On start, uses a timer callback to invoke change LED commands on a loop until skill is cancelled or times out. The functionality is similar to the `MostlyHarmlessSkill`, but the implementation is different.

HelloWorldSkill

On start, Misty loops through head and arm movement, audio playback, and image changes to greet the world.

HelloAgainWorldSkill

On start, Misty loops through head and arm movement, audio playback, and image changes to greet the world. The functionality is similar to the `HelloWorldSkill`, but the implementation is different.

LookAroundSkill

Demonstrates how to use timer callbacks in .NET skills. On start, the skill registers timer callback such that Misty receives randomized head movement, arm movement, and change LED commands.

InteractiveMistySkill

Demonstrates how to register, unregister, and handle events in your .NET skill code, and shows some of the different ways to listen for data from events and callbacks. Touch Misty's head and press her bump sensors to see her express different emotions.

HelloLocomotionSkill

Demonstrates how to use basic driving commands and handle data from Misty's hazard system in your .NET skills. On start, Misty drives around the room and avoids obstacles. Additionally, this skill uses functionality from many of the other sample skills to give Misty more personality.

ForceDriving

Demonstrates how to use the .NET SDK to build skills that have Misty engage with her environment in different and interesting ways. On start, the skill listens for data from Misty's time-of-flight (ToF) sensors. Place your hand in front of a range ToF sensor to make Misty drive in the opposite direction. The closer you bring your hand, the faster Misty moves. This skill uses functionality from many of the other sample skills to give Misty more personality.

Creating a New Skill

In addition to several pre-built C# skills, the `SkillLibrary` project provides a template for writing your own skills and deploying them via the `IntroSkillsTask` background application. Follow the steps below to use this template:

1. Make a copy of the `SkillTemplate.cs` file in the `SkillLibrary` project. Save it with a new name. This will be the name of your skill in Skill Runner.
2. Update the `NativeRobotSkill` implementation with your skill's name, a new unique GUID, and other optional fields as desired.
3. Add your skill code to the `OnStart()`, `OnCancel()`, and `OnTimeout()` methods. Save your changes.
4. In the `IntroSkillsTask` project, change the skill reference to your new skill in the `Startup.cs` file. Follow steps 3 and 4 in the Hello World with .NET guide to wrap the `StartupTask.cs` background application around your custom skill and deploy it to Misty.

Simulator

In addition to the library of skills you can install on Misty, the `IntroSkills` solution includes an example of a skill simulator built with Misty's .NET SDK libraries. The `SimulatedMisty` class in the `Simulator` project is an example of an event and command processing simulator that can be modified to create a custom simulated environment for testing skills without a robot.

> **Note:** The simulator is currently a work-in-progress, and is not expected to work with all skills at this time. It is included in the `IntroSkills` sandbox as an exercise for developers interested in experimenting with the mock robot capabilities of the .NET SDK in their own applications.